

SOUL FORGE

开发者技术文档

一款使用 Python 和 Pygame 制作的 2D 潜行平台跳跃游戏
开发人员: 1820251084 宋理、1820251088 颜键杨

内容

一.	项目概述	4
(一)	技术栈	4
二.	文件与目录结构	4
三.	源码运行指南	5
四.	游戏架构：状态机	5
(一)	状态列表	5
(二)	显著的转换逻辑	6
五.	核心系统	6
(一)	玩家系统	7
1.	关键常量	7
2.	移动机制	7
(二)	摄像机系统	7
(三)	关卡构建	8
1.	Warden 终端	8
2.	结局终端	8
(四)	终端与黑客小游戏	9
1.	终端 (hackTerminal.py)	9
2.	黑客小游戏(hackMinigame.py)	9
(五)	大门	9
(六)	基础敌人	10
1.	寻路	10
(七)	The Warden	10
1.	对话系统	11
2.	追逐 AI - 基于网格的 A*	11
(八)	传说与日志	11
(九)	存档系统	12
1.	存档文件位置	12
2.	自动存档	12
(十)	音频系统	13
1.	音效 (SFX)	13
2.	背景音乐与速度变换 (BGM & Speed Shift)	13
(十一)	贴图集与动画	13
六.	资源加载与 resource_path() 函数	14

七.	构建 .exe 应用程序.....	14
(一)	main.spec	14
(二)	build.bat	15
(三)	产生一个新构建.....	15
(四)	测试全新构建.....	15
八.	已知的怪癖、漏洞与陷阱.....	15
九.	额外说明	16
(一)	语言与翻译.....	16
(二)	AI 使用.....	16

一. 项目概述

《Soul Forge》是一款使用 Python 和 Pygame 开发的 2D 潜行平台跳跃游戏。玩家操控第 73 号灵魂 Marquette Verne，探索大型多房间地图、避开巡逻敌人、破解终端以开启大门、收集传说碎片（Lore fragments），并最终对决最终 Boss——Warden Gerard Gatewald。游戏提供叙事选择，导向两种不同的结局。

代码库组织为单进程的 Pygame 应用：main.py 中的主循环驱动全局状态机，而各个玩法系统（玩家移动、敌人、Boss、终端、音频、贴图、存档）存放在各自的模块中，并由该主循环统一调度。

（一） 技术栈

- Python 3（基于特定虚拟环境开发）
- Pygame —— 渲染、输入、音频基础组件、矩形碰撞（Rects & Collisions）
- NumPy —— 仅在 **audioManager.py** 中用于背景音乐（BGM）的实时重采样（报警时的音乐加速效果），以及在 **warden.py** 中用于对图像进行灰度化处理
- PyInstaller —— 用于将整个游戏打包为 Windows 独立 .exe 文件以便分发
- Bosca Ceoil —— 用于制作游戏的背景音乐与音效

二. 文件与目录结构

文件/文件夹	职责说明
main.py	程序入口点。持有游戏主循环、状态机、UI/菜单渲染，以及其他所有系统之间的逻辑衔接。
player.py	玩家物理逻辑：移动、重力、跳跃、土狼时间（Coyote time）、滑墙与墙跳（Wall jump）、攀爬（Mantling）及动画。
enemy.py	巡逻守卫 AI：巡逻/追逐/返回状态、视线检测（LOS）、基于 BFS 的路点寻路。
warden.py	Boss 角色：对话系统、A* 寻路/追逐逻辑、战败姿态及贴图渲染。
buildLevel.py	声明式关卡数据 —— 房间、平台几何体、终端/门/敌人/传说钥匙的摆放，以及组装新关卡的 build_level() 工厂函数。
hackTerminal.py	终端对象：黑客入侵状态、倒计时器、报警行为、基于贴图的视觉状态、存档/读档。
hackMinigame.py	具体的黑客小游戏：网格解谜（寻找钥匙、避开伪装块并到达出口）。

door.py	大门系统：当指定的终端被破解后自动开启，具备延迟与平滑滑动动画。
camera.py	房间吸附摄像机：房间切换时具备平滑的线性插值（Lerp）过渡效果。
checkpointManager.py	检查点区域：触发进入时自动存档的 Rect 区域（具体逻辑位于 main.py）。
lore.py	日志/传说系统：可收集的 LoreKey 对象、追踪已解锁章节的 JournalManager 以及所有章节文本。
saveManager.py	持久化存档数据（3 个存档槽）为 JSON 文件，以及菜单中通用的 Button UI 类。
audioManager.py	音效 + 音乐播放，包括警报期间背景音乐的自定义音调/速度变调效果。
spriteSheet.py	通用贴图集切割器以及玩家/敌人/终端使用的简单基于帧的 Animation 辅助类。
assets/	存放所有使用的资源，各自独立成文件夹（sprites/ 与 music/）。

注意：每一个从 assets/ 加载图像或声音文件的模块都需要第 6 节中描述的 resource_path() 辅助函数——这是添加新内容时最需要牢记的一点。

三. 源码运行指南

依赖项（必须安装）：

```
pip install pygame numpy
```

运行游戏：

```
python main.py
```

游戏窗口设定为 1000 × 750，目标帧率为 60 FPS（参见 main.py 顶部的 SCREEN_WIDTH / SCREEN_HEIGHT / FPS）。

四. 游戏架构：状态机

main.py 围绕单个全局变量 current_state 构建，其值为文件顶部定义的一组固定字符串常量。主循环根据该值分发更新逻辑与绘制逻辑。没有正式的状态机类——状态转换只是散布在事件处理和更新逻辑中的 current_state 普通赋值操作。

（一） 状态列表

状态名称	职责说明
MENU	标题主菜单界面

LEVEL_SELECT	存档选择界面
PLAYING	正常游戏玩法 —— 移动、敌人、终端、检查点
PAUSED	暂停遮罩；游戏逻辑冻结
HACKING	进入终端的小游戏。通过 <code>hackReturnState</code> 记录退出时应返回的状态
GAME_OVER	玩家被普通敌人或 Warden 捕获
JOURNAL	全屏传说日志，可从暂停菜单进入
READING	刚解锁的单个传说章节，拾取时立即显示
WARDEN_DIALOGUE	Boss 战场的打字机风格对话框（用于开场和战败文本）
WARDEN_INTRO	脚本化的渐变/传送过场，将玩家置入 Boss 战场
WARDEN_OUTRO	Warden 被击败后的脚本化闪烁/传送过场，将两个角色移至结局房间
ENDING_CHOICE	两个结局终端生效；破解任意一个将锁定该结局
ENDING_FADE	做出选择后渐变至黑屏
ENDING_EPILOGUE	每个结局独有的打字机风格尾声文本
ENDING	最终结算界面（时长、结局标签）

（二） 显著的转换逻辑

- **PLAYING → HACKING**：当玩家重叠终端交互区并按下 E 键时触发；返回状态保存在 `hackReturnState` 中（非硬编码），确保相同的按键处理适用于 **PLAYING** 与 **ENDING_CHOICE**。
- **HACKING 退**：可通过手动（再次按 E）或在破解后等待短延迟 `exitDelay` 自动退出。两条路径均通过 `hackReturnState` 分流，因此破解结局终端能正确返回 **ENDING_CHOICE** 而非落回 **PLAYING**。
- **击败 Warden**（`hackedCount >= 8` 且收集所有 Warden 传说）：设置 `current_state = WARDEN_OUTRO`，播放短过场后进入 **WARDEN_DIALOGUE** 状态，并将 `warden.dialogue_lines` 替换为 **DEFEAT_LINES**。
- **结束战败对话**：设置 `current_state = ENDING_CHOICE` 并调用 `enterEndingChoice()` 渐显两个结局终端。
- **world_timer** 仅在 **PLAYING** / **HACKING** / **WARDEN_INTRO** / **WARDEN_OUTRO** 状态下推进，明确排除 **ENDING_CHOICE**，因此在做最终决定时不会计时。

五. 核心系统

如前所述，该游戏本身采用多个不同的核心系统来运行完整游戏，这些系统各自独立划分为不同的模块。

（一） 玩家系统

`player.py` 实现了一个完善的平台跳跃控制器：基于加速度的水平移动、重力、土狼时间、带加速梯度的滑墙、墙跳以及墙面攀爬。

1. 关键常量

常量名	数值	含义说明
GRAVITY	0.6	每帧增加的垂直速度 <code>vel_y</code>
JUMP_STRENGTH	-13	跳跃时的初始向上速度
SHORT_HOP_CUTOFF	-4	提前松开跳跃键会限制 <code>vel_y</code> 至此值，实现蓄力/短跳
ACCEL / FRICTION	0.8 / 0.8	水平加速度 / 地面摩擦力系数
MOVE_SPEED	5	水平移动速度上限
COYOTE_TIME_MAX	5 frames	走出平台边缘后仍允许跳跃的宽限期
MAX_FALL_SPEED / MAX_FAST_FALL_SPEED	50 / 200	终端下落速度上限（正常 vs 按下键下落）
wallJumpForceX / wallJumpForceY	10 / -12	墙跳时施加的冲量
maxSlideSpd / fastSlideSpd	10.0 / 25.0	墙面下滑速度上限（正常 vs 下落变体）

2. 移动机制

- 土狼时间 (Coyote time)：在地面时刷新 `coyoteTimer`，离开地面后倒计时；`jump()` 检查 `coyoteTimer > 0` 作为 `on_ground` 的替代条件
- 滑墙 (Wall slide)：仅在玩家处于空中、贴墙、下落并按住贴墙方向键时触发；下滑速度上限随 `slideTimer` 运行时间分三级递增（慢抓 → 中速 → 快速）
- 墙面攀爬 (Wall mantle)：若玩家下滑时头部高于墙体边缘，`mantleClimb()` 会自动将玩家拉上墙头而非继续下滑
- 墙跳 (Wall jump)：若玩家按住反向键会施加更强水平冲量；同时设置短时间 `wallLockout` 防止立即重新抓墙
- 碰撞检测 (`moveX` / `moveY`)：采用固定步长 Sub—stepping (`MAX_STEP = 10px`) 推进，防止高速运动下穿过薄平台
- 动画：`walkAnim` 在处于地面且移动时播放；处于静止 (Idle) 状态时，静止帧会通过正弦函数 (`math.sin(idleTimer / 15.0)`) 进行形变/挤压，这纯粹是一种呼吸风格的视觉效果饰品，而非真正的动画帧

（二） 摄像机系统

`camera.py` 是一个基于房间吸附的轻量摄像机。每个房间为一个固定的 `pygame.Rect`；`targetRoom()` 设置目标位置，`update()` 每帧线性插值平滑移动

摄像机。切换期间 `isTrans` 为 `True`，`main.py` 利用此标志冻结玩家输入与敌人 AI，防止摄像机移动时发生逻辑更新。

`apply(rect)` 实现世界坐标到屏幕坐标的转换：直接将世界矩形按摄像机当前位置偏移。

（三） 关卡构建

`buildLevel.py` 主要是声明式数据：包含 `PUZZLES` 列表（手工设计或随机生成的黑客入侵小游戏布局）、`WAYPOINTS` 图字典，以及按房间分组的平台矩形。

每当调用 `build_level(level)` 时（例如在开始新游戏、读取存档或重置关卡时），它都会组装并返回一组全新的游戏对象：

- `rooms` —— 关卡中所有的实体碰撞矩形（目前有 25 间客房，索引为 0 - 24）
- `platforms` —— 关卡中所有的实体碰撞矩形
- `terminals` —— 通过 `copy.deepcopy(PUZZLES[i])` 从 `PUZZLES` 列表条目构建的终端实例（采用深拷贝，因此多个终端绝不会共享可变的小游戏状态）
- `doors` —— 大门实例，每个大门都关联了一组 `requiredTerminals`（必须全部被破解后大门才会开启）
- `enemies` —— 放置在固定坐标上的敌人实例，包含巡逻范围和共享的 `WAYPOINTS` 路径图
- `loreKeys`, `checkPoints` —— 可收集物 / 触发存档的对象
- `warden` —— 仅在房间列表足够长、包含 Boss 战场（`WARDEN_ARENA_ROOMS`）时才会构建；其生成位置推导自第一个战场房间的矩形区域

1. Warden 终端

Warden 放置完毕后，系统会在固定的手动摆放位置

（`wardenTerminalPositions`）额外生成 8 个带有 `room = "warden"` 标签的终端。这些终端的小游戏布局是通过 `random.sample()` 从包含 16 个高难度谜题的池子（`PUZZLES[10:26]`）中随机抽取的。这意味着每次游玩时，Warden 战场中各个终端的具体谜题都会发生变化，但它们在战场中的位置保持固定。

2. 结局终端

列表中最后的两个终端使用 `room = "endingChoiceWarden"` 与 `room = "endingChoiceBreak"` 标签，同时拥有“无限”的时间限制（设置为 `99999.0`），因此它们绝不会在玩家做决定时超时失效。`main.py` 通过这两个房间字符串标识它们（参见 `main.py` 中的 `syncEndingTerminal()`），而非通过列

表索引位置。正是通过这种方式，特定结局的 UI 文本（参见第 8 节）才能根据 `t.room` 来进行匹配和关联。

（四） 终端与黑客小游戏

1. 终端 (`hackTerminal.py`)

`Terminal` 包装了一个 `hackingMinigame` 实例，并添加了周围面向游戏的行为：一个倒计时器，如果该计时器耗尽则会触发警报状态，破解后有一个短暂的退出延迟（因此 `HACKING` 状态不会瞬间切回游玩状态），以及针对其三种状态（默认/警报/已破解）基于贴图的视觉效果。

- `canInteract(playerRect)` — 玩家必须处于终端碰撞矩形向外扩展 `(40, 50)` 的范围内才能按下 `E` 键
- `updateTimer(dt, isActive)` — 只有当 `hasStarted` 为 `True`（即玩家实际上至少打开过它一次）后才开始倒计时；归零时它会设置 `alarmTriggered` 并提供一个 10 秒的 `alarmBuffer`，之后警报才可以被清除（如果稍后被破解）
- 旋转支持 — 终端可以旋转 `90/-90` 度，在这种情况下，它的矩形宽/高在构建时会互换，以便碰撞依然与旋转后的贴图对齐
- `saveDict()` / `loadDict()` — 仅持久化计时器/已破解/警报状态，不保存谜题布局，因为底层的 `hackingMinigame` 在加载关卡时会根据同一个 `PUZZLES` 条目确定性地重建

2. 黑客小游戏 (`hackMinigame.py`)

一个小型网格解谜游戏（默认 6 行 × 8 列）。玩家使用方向键/WASD 移动一个标记，必须到达一个钥匙图块，然后在携带钥匙的情况下到达一个出口图块。特定的图块是“虚拟”诱饵——第一次踩到诱饵时会对父终端施加时间惩罚（`dummyPenalty`，默认为 5 秒），如果该惩罚将计时器耗尽至零，则可以直接触发警报。

图块位置（`keyPos`、`exitPos`、墙壁、诱饵）要么在 `PUZZLES` 的每个谜题中显式提供，要么当一个谜题条目仅指定了 `dummyCount` 时，通过 `getRandomOpenTile()` 进行随机化生成。

（五） 大门

一旦一个大门所有要求的终端（`requiredTerminals`）都报告 `isHacked == True`，该大门便会在 `closedPos` 和 `openPos`（每个大门相对于其关闭位置的偏移量）之间平滑移动。在“满足要求”和大门实际开始移动之间，有一个可配置的 `openDelay`（默认为 1 秒），加上一个固定的 `moveSpd`，用于每帧将 `currentPos` 朝 `targetPos` 插值移动。

- `forceClose()` / `releaseForceClose()` —— 被 Warden 的开场序列使用，用于在过场动画中间无视终端状态强行封闭一扇大门
- `syncToTerminalState()` —— 在加载关卡时调用一次，因此在一个之前的存档中其终端已被破解的大门，会在加载时直接卡入开启位置，而不是播放开启滑动动画

（六）基础敌人

`enemy.py` 实现了一个三状态巡逻守卫：PATROL、CHASE 和 RETURNING。

状态	行为
PATROL	默认的待机行为（在实际中移动模式存在于此类之外—— <code>patrol_range/start_x</code> 限制了它的游荡）
CHASE	由视线或 <code>forceChase</code> 触发（每当任何终端的警报激活时由 <code>main.py</code> 设置，所以关卡上的每个敌人在警报期间都会带有敌意）。如果可见则直接移向玩家，或者每隔 <code>REPATH_INTERVAL_FRAMES</code> 帧使用关卡的路点图重新向最后已知位置寻路
RETURNING	一旦 <code>chaseTimer</code> （一个记忆窗口， <code>CHASE_MEMORY_FRAMES = 300</code> 帧，约 5 秒）在未重新发现玩家的情况下到期时进入。它会向自己的生成点寻路返回；如果无法在 <code>MAX_RETURNING_FRAMES</code> 内找到返回的路线，则回退至普通巡逻状态

1. 寻路

敌人沿着一个共享的、手动编写的 WAYPOINTS 图（定义于 `buildLevel.py` 中）寻路，使用广度优先搜索（`update_path_to()`，在本文档中被截断但在 `enemy.py` 中存在）而不是自由形式的寻路——节点通过距离目标的远近来查找，并通过每个节点的邻居列表相连。如果敌人反复无法向某个节点推进，节点可以被暂时阻挡（`blocked_nodes`），迫使重新规划路线；阻挡在 `NODE_BLOCK_DURATION_FRAMES`（约 5 秒）后到期。

视线（`check_line_of_sight`）分为两部分：用于近距离侦测的简单距离检查，以及沿敌人当前面朝方向延伸的矩形重叠“视野锥”（实际上是一个矩形，`get_vision_rect()`），通过针对平台几何体的实际射线投射（`hasLOStoPlayer`，通过 `pygame.Rect.clipline`）进一步限制，所以敌人不能透视墙壁。

（七）The Warden

`warden.py` 是单一的首领角色 Gerard Gatewald。一旦战斗开始，它结合了脚本化的对话系统与完整的 A* 寻路追逐。

1. 对话系统

`dialogue_lines` 是一个一次推进一行的字符串列表；`tickDialogue()` 以 `charSpeed`（每秒 60 个字符）逐个字符显示当前行，并在完全显示后经过短暂停顿自动推进。`tickSkip()` 实现了一个长按跳过手势（按住 Y 持续 `skipHoldTime = 1` 秒），该功能仅在 `canSkip` 为 `True` 时启用——只要玩家在之前的会话中看过开场对话（通过 `SaveManager.markWardenIntroSeen()` 持久化），该值就会被设置，从而使回归玩家不被迫重新完整观看整个开场演讲。

相同的对话系统既可用于开场演讲（在 `__init__` 中设置 `dialogue_lines`），也可用于战败演讲（在 Warden 被击败后调入 `main.py` 中的 `DEFEAT_LINES`）。

2. 追逐 AI - 基于网格的 A*

与巡逻敌人手动编写的路点图不同，Warden 使用完整的 A* 搜索

（`aStarPathfinding()`）在每次重新计划时（`findPath()`），基于 Warden 与玩家的当前位置周围动态生成的新网格进行寻路。网格分辨率为 `PATH_CELL_SIZE = 60px`，限制为每个轴最高 `PATH_MAX_GRID_CELLS = 40` 个单元格（如果搜索区域将超出此限制，则扩大单元格大小，以限制最坏情况下的搜索成本）。允许对角线移动，但明确禁止抄近道穿过两个正交阻挡的单元格。

然后，利用贪婪的视线简化对原始单元格路径进行平滑处理（`smoothPath()`）：从每个点出发，它会尽可能向前方查看一条没有阻挡的线段（`clearLine()`，针对放大的平台矩形的 `clipline`），并直接跳至那里，跳过中间路点。寻路每 `PATH_REPLAN_FRAMES`（20 帧）重新运行一次，或者如果 Warden 卡住（`stuckFrames >= 10` 帧保持近乎零移动且仍然远离目标）会立即重新运行，在这种情况下，在恢复正常寻路前它也会侧向挪动一个短暂的 `unstuckFrames` 窗口。

移动本身是直接朝着目标的速度（不像玩家或巡逻敌人那样基于物理/重力）——Warden 实际上是在飞行/悬停，这与其外观上的悬停摆动（`self.hover`，纯粹视觉效果，在 `draw()` 中为眼睛贴图的正弦波摆动提供数据）相一致。

（八） 传说与日志

`lore.py` 将每一篇可收集的传说定义为 `JOURNAL_CHAPTERS` 中的静态文本（通过章节 ID 键入，例如 `"ch3_2"`），一个 `LoreKey` 拾取物类（一个摆动的发光可收集物），以及 `JournalManager`，它会跟踪已解锁的章节 ID，并以固定的阅读顺序（`self.order`）展示，筛选至仅保留已解锁的条目（`get_visible_chapters()`）。

Warden 故事线的章节（`ch4_1`, `ch4_2`, `ch5_1`, `ch5_2` —— 在 `main.py` 中被集合为 `WARDEN_LORE_IDS`）被特殊对待：它们是触发 Warden 战败的必要条件（与

破解 8 个 Warden 竞技场终端一起），并且 `main.py` 中的 `relockWardenChapters()/restoreLoreKeys()` 函数每当关卡被重建（例如在加载时）就会重新锁定并重新同步这些特定章节，以至于一个传说钥匙不会以与战斗完成要求不一致的方式永久保持“已收集”状态。

`NotificationManager` 是一个不相关的小型浮动通知队列（淡出文本消息）；它存在于 `lore.py` 中，但目前未连接到 `main.py` 中的任何调用站点。

（九） 存档系统

`saveManager.py`'s 的 `SaveManager` 类保留了一个包含 3 个固定存档槽位的单一 JSON 文件。每个槽位存储：关卡，房间，`player_x/player_y`，一个 `exists` 标志，消耗的世界计时器，已解锁的日志章节列表，已达成的结局，是否已观看 Warden 开场对话，以及每一个终端和敌人状态的完整快照（通过每个对象的 `saveDict()/loadDict()`）。

1. 存档文件位置

`SAVE_FILE` 不是相对路径——它通过 `main.py` 中的一个微小 `get_save_dir()` 辅助函数解析为按用户、按操作系统的应用数据目录：

```
def get_save_dir(app_name="StealthPlatformer"):
    if sys.platform == "win32":
        base = os.environ.get("APPDATA", os.path.expanduser("~"))
        elif sys.platform == "darwin":
            base = os.path.join(os.path.expanduser("~"), "Library", "Application
Support")
        else:
            base = os.environ.get("XDG_DATA_HOME",
os.path.join(os.path.expanduser("~"), ".local", "share"))
        save_dir = os.path.join(base, app_name)
        os.makedirs(save_dir, exist_ok=True)
        return save_dir

SAVE_FILE = os.path.join(get_save_dir(), "save_data.json")
```

在 Windows 上，它会被解析为 `%APPDATA%\StealthPlatformer\save_data.json`。这一点特别重要，因为打包的 `.exe`（第 7 节）不是从一个稳定、可写甚至一致的工作目录运行的——如果在可执行文件旁边写入（或更糟的是，写入每次运行后都被删除的 `PyInstaller`——`onefile` 临时提取文件夹），将默默地破坏保存。任何未来的存档相关代码必须通过这个相同的路径写入，绝不可使用裸的相对文件名。

2. 自动存档

存档自动发生在这两处，均来自于 `main.py`：进入 Checkpoint 矩形（`checkCheckpoints()`，它还会触发短促的“SAVED”淡出通知），以及

WARDEN_INTRO 完成 / 正常游戏循环退出时。没有手动存档按钮——进度是检查点驱动的。

（十） 音频系统

`audioManager.py` 封装了 Pygame 的混音器，具有两项独立责任：即发即弃的 SFX 播放，以及一个支持即时速度更改的单一流式背景音乐轨道（用于警报状态音乐加速）。

1. 音效 (SFX)

所有音效在构建时被一次性预加载至 `self.sounds`（名称 → Sound 字典）。`playSfx()` 从一个共享池（`pygame.mixer.find_channel(True)`）在任何空闲通道上播放大部分声音，因此重叠的 SFX 不会互相切断，除了“walk”的脚步声，它被传送到单独专注的 `walkChannel`，以免过快的脚步声刷屏占用多个通道。

2. 背景音乐与速度变换 (BGM & Speed Shift)

并非使用 Pygame 的内置音乐流（没有逐曲目速度控制），BGM 被完整作为 NumPy 数组（`pygame.sndarray.array`）加载进内存，并作为手动重新采样的 Sound 回放。`setBgmSpeed()` 通过对数组重新采样（`resampleArray()`，被速度因子缩放的简单最近邻索引选择）来更改播放速率，并立即从正确的样本偏移量重新启动播放，所以更改是无缝的，而不是从 0:00 重新启动曲目。由 `main.py`（针对 `audioManager.bgmSpeed` 检查 `alarmActive`）驱动的“警报期间音乐加速 25%”效果正是由它驱动的。

`playBgm()` 对每首曲目具有幂等性——用相同文件路径重复调用它（如同 `main.py` 基于 `current_state/currRoom` 在每一帧所做的）在首次调用后是无操作的，因此正确的曲目实际只在更改时才重新加载。

（十一） 贴图集与动画

`spriteSheet.py` 是一个供 `player.py`、`enemy.py` 和 `hackTerminal.py` 使用的微小、通用的实用程序。`spriteSheet(filePath, frameWidth, frameHeight, scale)` 一次性加载图像并按固定单元格大小将其切片为一个平面的帧列表（行主序），带有可选的统一缩放。`animation(frames, frameDuration, loop)` 是一个简单的帧计时器，以固定速率在帧列表中推进，可以循环或停留在最后一帧。

注意： 这就是 `resource_path()` 修复（第 6 节）集中所在的文件——修复此处的图像加载，无需逐个触碰那些文件即可自动修复每一个调用者（玩家、敌人、终端）。

六. 资源加载与 `resource_path()` 函数

以类似 “assets/sprites/Chud-Sheet.png” 的纯相对路径从磁盘加载文件的任何代码，仅当当前工作目录恰好是项目根目录时才能正常运行——在编辑器里从项目文件夹运行 `python main.py` 时是真的，但一旦游戏打包进独立 `.exe`（第 7 节），这便无法保证了，因为它可以通过从任何文件夹双击来启动。

在整个代码库中使用的修复方法是一个微小的辅助函数，在使用它的每个文件里（`spriteSheet.py`, `warden.py`, `audioManager.py`）被复制：

```
def resourcePath(relativePath):
    basePath = getattr(sys, "_MEIPASS",
os.path.dirname(os.path.abspath(__file__)))
    return os.path.join(basePath, relativePath)
```

`sys._MEIPASS` 是 PyInstaller 在打包的 `--onefile` 可执行文件中运行时才注入的一个特殊属性，指向捆绑包被提取出的临时文件夹。从纯源码运行（不存在此类属性）时，它回退到包含当前 `.py` 文件的目录。每一个资产路径在交给 `pygame.image.load()`、`pygame.mixer.Sound()` 等之前都应该被包装在此中。

注意：对于那些尚未通过 `spriteSheet.py` 或 `audioManager.playBgm()/playSfx()`（它们已经在内部包装了它）汇聚的每一个新的资产加载调用点，这必须手动完成。在任何其他地方未经此包装添加一个裸露的 `pygame.image.load("assets/...")`，在源码中可以正常工作，但在打包后的 `.exe` 中会默默崩掉。

七. 构建 `.exe` 应用程序

打包由 PyInstaller 完成，由检入的 `main.spec` 文件加上一个小型的 `build.bat` 封装器驱动（由于项目的依赖驻留在外部虚拟环境而非全局 Python 安装中，`.bat` 直接调用该虚拟环境中的 Python，而不是依赖 `PATH` 上的 `pyinstaller`）。

（一） `main.spec`

```
a = Analysis(
    ['main.py'],
    pathex=['<project folder>'],
    datas=[('<project folder>\\assets', 'assets')],
    ...
)

exe = EXE(
    pyz, a.scripts, a.binaries, a.zipfiles, a.datas, [],
    name='main',
    console=False,    # windowed, no console popup
    ...
)
```

datas 条目正是将整个 **assets/** 文件夹（包含其 **music/bgm** 和 **music/sfx** 子文件夹）打包进可执行文件的原因——PyInstaller 递归复制引用的文件夹，所以无需针对每个子文件夹设置条目。

（二） build.bat

```
@echo off
"<path to venv>\Scripts\python.exe" -m PyInstaller main.spec --distpath
"output/dist"
--workpath "output/build" --noconfirm -clean
pause
```

- **--noconfirm** 跳过交互式的“覆盖现有构建？”提示，否则如果以非交互方式运行会静默中止重新构建。
- **--clean** 擦除 PyInstaller 的内部缓存，因此重新构建不会不小心重复使用前次运行里过时的编译工件

（三） 产生一个新构建

在项目根目录双击 **build.bat**。完成的可执行文件将写入 **output/dist/main.exe**。

（四） 测试全新构建

推荐测试前将 **main.exe** 从 **output/dist** 拷出至不相干的文件夹（如桌面）——在项目文件夹内部运行可能会掩饰游戏意外仍然依赖坐落在源码旁的散落文件（相对正确打包/重定向版本：通过 **resource_path()** 获取资产、通过 **%APPDATA%** 获取保存数据）的 bug。

八. 已知的怪癖、漏洞与陷阱

一份运行时非显而易见行为和以往漏洞的列表，在扩展此代码库时值得铭记：

- **hackReturnState** 的存在专门是为了 **HACKING** 状态（从 **PLAYING** 或 **ENDING_CHOICE** 进）在退出时知道应当返回哪个状态——主循环中的手动退出按键处理程序和自动倒计时过期退出均必须读取此变量而非硬编码为 **PLAYING**，不然骇入一个结局终端将错误地把玩家抛回普通游戏模式。
- 两个结局终端通过它们的房间字符串（**"endingChoiceWarden"** / **"endingChoiceBreak"**）而不是它们在列表中的位置或对象身份被识别——任何需要区分它们的 UI 文本或逻辑（例如“成为典狱长”对比“打破循环”提示，或在骇入完毕屏幕显示“已达成结局”取代“允许访问”）均应以 **activeTerminal.room** 为键

- `world_timer` 明确地把 `ENDING_CHOICE`（以及 `WARDEN_INTRO` / `WARDEN_OUTRO` 过场）从计时中排除了——如果未来有个状态同样需要计时器被暂停，必须将其加进主循环中的该同一排除元组。
- 任何新的 `pygame.image.load(...)` 或 `pygame.mixer.Sound(...)` 调用必须被包装在 `resourcePath()`（第 6 节）中，否则即使源码下没问题，一旦被打包进 `.exe` 便将静默失败。
- 存档文件的位置特定于 OS 用户（Windows 上的 `%APPDATA%`），且不与项目文件夹或 `.exe` 的位置捆绑——源里运行该游戏与在同一台机器上运行构建好的 `.exe` 将共享同样的保存数据，这是设计使然。
- PyInstaller 编译并不是自动全新的：若无 `--noconfirm --clean`，若以非交互式（如双击 `.bat`）执行构建，便可能在覆写过时的可执行文件时静默失败，留着你调试那个看起来“没起作用”的改动代码。如果你的修复看上去未工作，永远将 `exe` 的修改时间戳拿去对比你最新的源头编辑。
- 敌方 AI 众所周知偶有卡上岩架的问题，这是目前为止还没有解决的一个已知漏洞。

九. 额外说明

（一） 语言与翻译

该游戏目前的呈现采用英文作为其主要语言，可见于贯穿整个 UI 元素、对话及通知中。由于本游戏这两名开发者在读写上首先使用的是英文，我们在制造这游戏里任何事物时理所应当第一位并最首要地预设为英文。

翻译到中文语言可能是办得到的，虽然这势必需要一些额外时间以及一套能容许所谓翻译的新系统，再者说来，必需造出一套新的文件系统（`lang.json`）。

（二） AI 使用

为了透明起见，本游戏确实有些 AI 的使用，即在写代码方面。确切地说是在敌方 AI 里，某些代码的使用直接出自 AI。除此点以外，AI 的使用被保留在最小程度内。